

Lab: Distributed Application + Zero Trust Security with Istio on AKS

1. Lab Overview

In this lab, you will deploy a distributed e-commerce-style application to Azure Kubernetes Service (AKS) and secure service-to-service communication using Istio.

The lab focuses on Zero Trust principles applied to microservices:

- Never trust the internal network by default.
- Require strong workload identity.
- Encrypt service-to-service communication with mutual TLS.
- Apply least-privilege service access.
- Validate allowed and denied traffic.
- Observe the system through sidecar proxies and Kubernetes events.

This lab is designed for an Azure-based environment using AKS and the managed Istio add-on.

2. Business Scenario

You are building a simplified e-commerce platform with these logical services:

```
store-front
order-service
payment-service
inventory-service
shipping-service
notifications-service
```

The intended business flow is:

```
Customer
↓
store-front
↓
order-service
↓
payment-service
↓
inventory-service
```

```
↓
shipping-service
↓
notifications-service
```

From a Zero Trust perspective, the goal is not only to deploy the services. The goal is to control exactly which service is allowed to call another service.

For example:

store-front	→ order-service	Allowed
store-front	→ payment-service	Denied
order-service	→ payment-service	Allowed
order-service	→ inventory-service	Allowed
payment-service	→ inventory-service	Denied
inventory-service	→ shipping-service	Allowed
shipping-service	→ notifications-service	Allowed

3. Learning Objectives

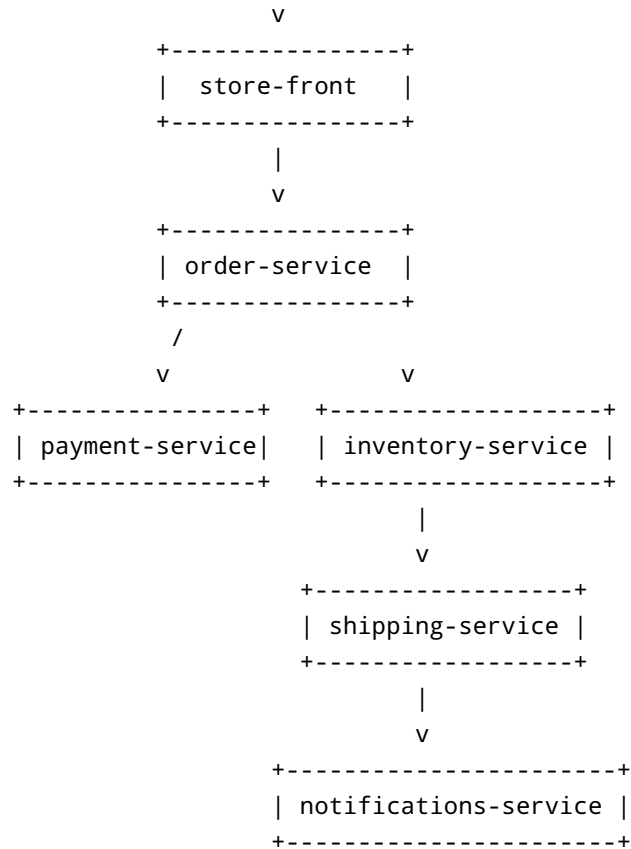
By the end of this lab, you will be able to:

1. Enable the Istio service mesh add-on on an AKS cluster.
2. Inject Istio sidecars into application workloads.
3. Deploy a distributed microservices-style application.
4. Validate baseline service-to-service communication.
5. Enforce mTLS using Istio `PeerAuthentication`.
6. Implement namespace-level default deny.
7. Implement least-privilege service authorization using `AuthorizationPolicy`.
8. Test allowed and denied communication paths.
9. Validate that non-mesh traffic is rejected.
10. Understand how Zero Trust applies to microservices in AKS.

4. Architecture

4.1 Logical Architecture

```
+-----+
|  Customer  |
+-----+
|
```



4.2 Security Architecture

Each microservice runs with its own Kubernetes ServiceAccount.

Istio uses these service accounts as workload identities.

```

store-front-sa
order-service-sa
payment-service-sa
inventory-service-sa
shipping-service-sa
notifications-service-sa
  
```

The security controls are:

```

mTLS STRICT
+
Default Deny
  
```

+
Explicit Allow Policies

This means:

No identity → no access
No policy → no access
No mesh sidecar → no access

5. Prerequisites

You need:

- An Azure subscription.
- An AKS cluster.
- Azure CLI.
- kubectl.
- Sufficient permissions to modify the AKS cluster.
- AKS cluster credentials configured locally or in Azure Cloud Shell.
- A Linux-based AKS node pool.
- The AKS cluster must not already have a conflicting self-managed Istio installation.

Recommended minimum environment:

AKS node count: 2
VM size: Standard_D4_v4 or similar
Kubernetes version: supported AKS version
Region: centralus or your assigned Azure region

6. Lab Variables

Set your environment variables.

Replace the values according to your student environment.

```
export RESOURCE_GROUP="user20"  
export CLUSTER_NAME="myAKSCluster0325720758"  
export LOCATION="centralus"
```

```
export APP_NS="zt-ecommerce"
export OUTSIDE_NS="outside-mesh"
```

Validate your Azure account:

```
az account show -o table
```

Connect to AKS:

```
az aks get-credentials
--resource-group $RESOURCE_GROUP
--name $CLUSTER_NAME
--overwrite-existing
```

Validate access to the cluster:

```
kubectl get nodes
```

Expected result:

NAME	STATUS	ROLES	AGE	VERSION
aks-nodepool1-xxxxxxx-vmss000000	Ready	agent	...	v1.xx.x

7. Enable Istio Add-on on AKS

Check if the Istio service mesh add-on is already enabled:

```
az aks show
--resource-group $RESOURCE_GROUP
--name $CLUSTER_NAME
--query "serviceMeshProfile.mode"
-o tsv
```

If the output is:

```
Istio
```

then the add-on is already enabled.

If the output is empty or null, enable the Istio add-on:

```
az aks mesh enable
--resource-group $RESOURCE_GROUP
--name $CLUSTER_NAME
```

Verify again:

```
az aks show
--resource-group $RESOURCE_GROUP
--name $CLUSTER_NAME
--query "serviceMeshProfile.mode"
-o tsv
```

Expected output:

```
Istio
```

8. Verify Istio Control Plane

Get AKS credentials again:

```
az aks get-credentials
--resource-group $RESOURCE_GROUP
--name $CLUSTER_NAME
--overwrite-existing
```

Check the Istio control plane namespace:

```
kubectl get ns
```

Check the Istio control plane pods:

```
kubectl get pods -n aks-istio-system
```

Expected result:

NAME	READY	STATUS	RESTARTS	AGE
istiod-asm-x-xx-xxxxxxxxxx-xxxxx	1/1	Running	0	...

9. Get the Istio Revision

The AKS Istio add-on requires revision-based injection.

Get the installed Istio revision:

```
export ISTIO_REVISION=$(az aks show
  --resource-group $RESOURCE_GROUP
  --name $CLUSTER_NAME
  --query "serviceMeshProfile.istio.revisions[0]"
  -o tsv)

echo $ISTIO_REVISION
```

Example output:

```
asm-1-24
```

10. Create the Application Namespace

Create the namespace:

```
kubectl create namespace $APP_NS
```

Label the namespace with the Istio revision:

```
kubectl label namespace $APP_NS istio.io/rev=$ISTIO_REVISION --overwrite
```

Validate the namespace label:

```
kubectl get namespace $APP_NS --show-labels
```

Expected output should include:

```
istio.io/rev=asm-x-xx
```

11. Deploy the Distributed E-commerce Demo Application

This application is intentionally simple. Each service is represented by a lightweight HTTP service. The purpose of the lab is not the business logic itself, but the service-to-service security model.

Create the manifest:

```
cat <<'EOF' > zt-ecommerce-app.yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: store-front-sa
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: order-service-sa
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: payment-service-sa
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: inventory-service-sa
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: shipping-service-sa
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: notifications-service-sa
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: store-front
```



```

    labels:
      app: store-front
spec:
  replicas: 1
  selector:
    matchLabels:
      app: store-front
  template:
    metadata:
      labels:
        app: store-front
    spec:
      serviceAccountName: store-front-sa
      containers:
      - name: app
        image: hashicorp/http-echo:1.0
        args:
        - "-text=store-front: customer entry point"
        - "-listen=:8080"
        ports:
        - containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
  name: store-front
spec:
  selector:
    app: store-front
  ports:
  - name: http
    port: 8080
    targetPort: 8080
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-service
  labels:
    app: order-service
spec:
  replicas: 1
  selector:
    matchLabels:
      app: order-service
  template:
    metadata:
      labels:

```

```

        app: order-service
spec:
  serviceAccountName: order-service-sa
  containers:
  - name: app
    image: hashicorp/http-echo:1.0
    args:
    - "-text=order-service: order created"
    - "-listen=:8080"
    ports:
    - containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
  name: order-service
spec:
  selector:
    app: order-service
  ports:
  - name: http
    port: 8080
    targetPort: 8080
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: payment-service
  labels:
    app: payment-service
spec:
  replicas: 1
  selector:
    matchLabels:
      app: payment-service
  template:
    metadata:
      labels:
        app: payment-service
    spec:
      serviceAccountName: payment-service-sa
      containers:
      - name: app
        image: hashicorp/http-echo:1.0
        args:
        - "-text=payment-service: payment authorized"
        - "-listen=:8080"
        ports:

```

```

        - containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
  name: payment-service
spec:
  selector:
    app: payment-service
  ports:
    - name: http
      port: 8080
      targetPort: 8080
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: inventory-service
  labels:
    app: inventory-service
spec:
  replicas: 1
  selector:
    matchLabels:
      app: inventory-service
  template:
    metadata:
      labels:
        app: inventory-service
    spec:
      serviceAccountName: inventory-service-sa
      containers:
        - name: app
          image: hashicorp/http-echo:1.0
          args:
            - "-text=inventory-service: inventory reserved"
            - "-listen=:8080"
          ports:
            - containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
  name: inventory-service
spec:
  selector:
    app: inventory-service
  ports:

```

```

    - name: http
      port: 8080
      targetPort: 8080
  ---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: shipping-service
  labels:
    app: shipping-service
spec:
  replicas: 1
  selector:
    matchLabels:
      app: shipping-service
  template:
    metadata:
      labels:
        app: shipping-service
    spec:
      serviceAccountName: shipping-service-sa
      containers:
        - name: app
          image: hashicorp/http-echo:1.0
          args:
            - "-text=shipping-service: shipment created"
            - "-listen=:8080"
          ports:
            - containerPort: 8080
  ---
apiVersion: v1
kind: Service
metadata:
  name: shipping-service
spec:
  selector:
    app: shipping-service
  ports:
    - name: http
      port: 8080
      targetPort: 8080
  ---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: notifications-service
  labels:
    app: notifications-service

```

```

spec:
  replicas: 1
  selector:
    matchLabels:
      app: notifications-service
  template:
    metadata:
      labels:
        app: notifications-service
    spec:
      serviceAccountName: notifications-service-sa
      containers:
      - name: app
        image: hashicorp/http-echo:1.0
        args:
        - "-text=notifications-service: notification sent"
        - "-listen=:8080"
        ports:
        - containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
  name: notifications-service
spec:
  selector:
    app: notifications-service
  ports:
  - name: http
    port: 8080
    targetPort: 8080
EOF

```

Apply the manifest:

```
kubectl apply -n $APP_NS -f zt-ecommerce-app.yaml
```

Wait for the pods:

```
kubectl wait --for=condition=Ready pod --all -n $APP_NS --timeout=180s
```

Validate pods:

```
kubectl get pods -n $APP_NS
```

Expected result:

NAME	READY	STATUS	RESTARTS	AGE
store-front-xxxxxxxx-xxxxx	2/2	Running	0	...
order-service-xxxxxxxx-xxxxx	2/2	Running	0	...
payment-service-xxxxxxxx-xxxxx	2/2	Running	0	...
inventory-service-xxxxxxxx-xxxxx	2/2	Running	0	...
shipping-service-xxxxxxxx-xxxxx	2/2	Running	0	...
notifications-service-xxxxxxxx-xxxxx	2/2	Running	0	...

The important detail is:

```
READY 2/2
```

This means each pod has:

```
Application container  
+  
Istio sidecar proxy
```

12. Deploy Mesh-Aware Test Clients

To test service-to-service communication, deploy small curl clients using different service accounts.

Each test client represents a microservice identity.

```
cat <<'EOF' > zt-ecommerce-clients.yaml  
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: client-store-front  
  labels:  
    app: client-store-front  
spec:  
  replicas: 1  
  selector:  
    matchLabels:  
      app: client-store-front
```

```

template:
  metadata:
    labels:
      app: client-store-front
  spec:
    serviceAccountName: store-front-sa
    containers:
      - name: curl
        image: curlimages/curl:8.8.0
        command: ["sh", "-c", "sleep 365d"]
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: client-order
  labels:
    app: client-order
spec:
  replicas: 1
  selector:
    matchLabels:
      app: client-order
  template:
    metadata:
      labels:
        app: client-order
    spec:
      serviceAccountName: order-service-sa
      containers:
        - name: curl
          image: curlimages/curl:8.8.0
          command: ["sh", "-c", "sleep 365d"]
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: client-payment
  labels:
    app: client-payment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: client-payment
  template:
    metadata:
      labels:
        app: client-payment

```

```

    spec:
      serviceAccountName: payment-service-sa
      containers:
      - name: curl
        image: curlimages/curl:8.8.0
        command: ["sh", "-c", "sleep 365d"]
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: client-inventory
  labels:
    app: client-inventory
spec:
  replicas: 1
  selector:
    matchLabels:
      app: client-inventory
  template:
    metadata:
      labels:
        app: client-inventory
    spec:
      serviceAccountName: inventory-service-sa
      containers:
      - name: curl
        image: curlimages/curl:8.8.0
        command: ["sh", "-c", "sleep 365d"]
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: client-shipping
  labels:
    app: client-shipping
spec:
  replicas: 1
  selector:
    matchLabels:
      app: client-shipping
  template:
    metadata:
      labels:
        app: client-shipping
    spec:
      serviceAccountName: shipping-service-sa
      containers:
      - name: curl

```



```
image: curlimages/curl:8.8.0
command: ["sh", "-c", "sleep 365d"]
EOF
```

Apply the manifest:

```
kubectl apply -n $APP_NS -f zt-ecommerce-clients.yaml
```

Wait for the clients:

```
kubectl wait --for=condition=Ready pod -l app=client-store-front -n $APP_NS --
timeout=180s
kubectl wait --for=condition=Ready pod -l app=client-order -n $APP_NS --
timeout=180s
kubectl wait --for=condition=Ready pod -l app=client-payment -n $APP_NS --
timeout=180s
kubectl wait --for=condition=Ready pod -l app=client-inventory -n $APP_NS --
timeout=180s
kubectl wait --for=condition=Ready pod -l app=client-shipping -n $APP_NS --
timeout=180s
```

Validate:

```
kubectl get pods -n $APP_NS
```

The client pods should also show:

```
READY 2/2
```

13. Baseline Test Before Security Policies

At this point, Istio sidecars are injected, but we have not applied Zero Trust authorization policies yet.

Test from `store-front` identity to `order-service`:

```
kubectl exec -n $APP_NS deploy/client-store-front -c curl --
curl -sS http://order-service:8080
```

Expected result:

```
order-service: order created
```

Test from `store-front` identity directly to `payment-service`:

```
kubectl exec -n $APP_NS deploy/client-store-front -c curl --  
curl -sS http://payment-service:8080
```

Expected result:

```
payment-service: payment authorized
```

This is intentionally insecure.

Right now, any service can call any other service.

That is what we are going to fix.

14. Apply mTLS STRICT

Create a namespace-wide `PeerAuthentication` policy.

```
cat <<EOF > peer-authentication-strict.yaml  
apiVersion: security.istio.io/v1  
kind: PeerAuthentication  
metadata:  
  name: default  
  namespace: ${APP_NS}  
spec:  
  mtls:  
    mode: STRICT  
EOF
```

Apply it:

```
kubectl apply -f peer-authentication-strict.yaml
```

Validate:

```
kubectl get peerauthentication -n $APP_NS
```

Expected result:

NAME	MODE	AGE
default	STRICT	...

Test mesh-to-mesh traffic again:

```
kubectl exec -n $APP_NS deploy/client-store-front -c curl --  
curl -sS http://order-service:8080
```

Expected result:

```
order-service: order created
```

This still works because both workloads are inside the mesh and have Istio sidecars.

15. Validate That Non-Mesh Traffic Fails

Create a namespace without Istio injection:

```
kubectl create namespace $OUTSIDE_NS
```

Create a curl pod outside the mesh:

```
kubectl run outside-client  
-n $OUTSIDE_NS  
--image=curlimages/curl:8.8.0  
--command -- sleep 365d
```

Wait for the pod:

```
kubectl wait --for=condition=Ready pod/outside-client -n $OUTSIDE_NS --  
timeout=180s
```

Try to call `order-service` from outside the mesh:

```
kubectl exec -n $OUTSIDE_NS outside-client --  
  curl -sS --max-time 5 http://order-service.${APP_NS}.svc.cluster.local:8080  
|| true
```

Expected result:

```
Connection reset
```

or:

```
upstream connect error
```

or another connection failure.

This is good.

It means the service requires mTLS and does not accept plaintext traffic from non-mesh workloads.

16. Apply Default Deny

Now we will apply the most important Zero Trust step:

```
Deny everything by default.
```

Create the policy:

```
cat <<EOF > default-deny.yaml  
apiVersion: security.istio.io/v1  
kind: AuthorizationPolicy  
metadata:  
  name: default-deny  
  namespace: ${APP_NS}  
spec: {}  
EOF
```

Apply it:

```
kubectl apply -f default-deny.yaml
```

Test from `store-front` to `order-service`:

```
kubectl exec -n $APP_NS deploy/client-store-front -c curl --  
sh -c 'curl -s -o /dev/null -w "%{http_code}\n" http://order-service:8080'
```

Expected result:

403

This is correct.

The mesh now denies all traffic unless an explicit allow policy exists.

17. Apply Least-Privilege Authorization Policies

Now we will allow only the approved service-to-service paths.

Allowed paths:

```
store-front      → order-service  
order-service    → payment-service  
order-service    → inventory-service  
inventory-service → shipping-service  
shipping-service → notifications-service
```

Create the policies:

```
cat <<EOF > ecommerce-allow-policies.yaml  
apiVersion: security.istio.io/v1  
kind: AuthorizationPolicy  
metadata:  
  name: allow-store-front-to-order  
  namespace: ${APP_NS}  
spec:  
  selector:  
    matchLabels:  
      app: order-service  
  action: ALLOW  
  rules:  
  - from:  
    - source:
```

```

        principals:
        - cluster.local/ns/${APP_NS}/sa/store-front-sa
    ---
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: allow-order-to-payment
  namespace: ${APP_NS}
spec:
  selector:
    matchLabels:
      app: payment-service
  action: ALLOW
  rules:
  - from:
    - source:
      principals:
      - cluster.local/ns/${APP_NS}/sa/order-service-sa
    ---
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: allow-order-to-inventory
  namespace: ${APP_NS}
spec:
  selector:
    matchLabels:
      app: inventory-service
  action: ALLOW
  rules:
  - from:
    - source:
      principals:
      - cluster.local/ns/${APP_NS}/sa/order-service-sa
    ---
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: allow-inventory-to-shipping
  namespace: ${APP_NS}
spec:
  selector:
    matchLabels:
      app: shipping-service
  action: ALLOW
  rules:
  - from:
    - source:

```

```
    principals:
      - cluster.local/ns/${APP_NS}/sa/inventory-service-sa
---
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: allow-shipping-to-notifications
  namespace: ${APP_NS}
spec:
  selector:
    matchLabels:
      app: notifications-service
  action: ALLOW
  rules:
    - from:
        - source:
            principals:
              - cluster.local/ns/${APP_NS}/sa/shipping-service-sa
EOF
```

Apply the policies:

```
kubectl apply -f ecommerce-allow-policies.yaml
```

Validate:

```
kubectl get authorizationpolicy -n $APP_NS
```

Expected result:

NAME	ACTION	AGE
default-deny	ALLOW	...
allow-store-front-to-order	ALLOW	...
allow-order-to-payment	ALLOW	...
allow-order-to-inventory	ALLOW	...
allow-inventory-to-shipping	ALLOW	...
allow-shipping-to-notifications	ALLOW	...

18. Test Allowed Paths

18.1 store-front → order-service

```
kubectl exec -n $APP_NS deploy/client-store-front -c curl --  
  sh -c 'curl -s -o /tmp/out -w "%{http_code}\n" http://order-service:8080 &&  
  cat /tmp/out'
```

Expected result:

```
200  
order-service: order created
```

18.2 order-service → payment-service

```
kubectl exec -n $APP_NS deploy/client-order -c curl --  
  sh -c 'curl -s -o /tmp/out -w "%{http_code}\n" http://payment-service:8080 &&  
  cat /tmp/out'
```

Expected result:

```
200  
payment-service: payment authorized
```

18.3 order-service → inventory-service

```
kubectl exec -n $APP_NS deploy/client-order -c curl --  
  sh -c 'curl -s -o /tmp/out -w "%{http_code}\n" http://inventory-service:8080  
  && cat /tmp/out'
```

Expected result:

```
200  
inventory-service: inventory reserved
```

18.4 inventory-service → shipping-service

```
kubectl exec -n $APP_NS deploy/client-inventory -c curl --  
  sh -c
```



```
'curl -s -o /tmp/out -w "%{http_code}\n" http://shipping-service:8080 && cat /tmp/out'
```

Expected result:

```
200
shipping-service: shipment created
```

18.5 shipping-service → notifications-service

```
kubectl exec -n $APP_NS deploy/client-shipping -c curl --
  sh -c 'curl -s -o /tmp/out -w "%{http_code}\n" http://notifications-service:
8080 && cat /tmp/out'
```

Expected result:

```
200
notifications-service: notification sent
```

19. Test Denied Paths

19.1 store-front → payment-service

The store front should not call Payments directly.

```
kubectl exec -n $APP_NS deploy/client-store-front -c curl --
  sh -c 'curl -s -o /dev/null -w "%{http_code}\n" http://payment-service:8080'
```

Expected result:

```
403
```

19.2 store-front → inventory-service

The store front should not call Inventory directly.

```
kubectl exec -n $APP_NS deploy/client-store-front -c curl --  
sh -c 'curl -s -o /dev/null -w "%{http_code}\n" http://inventory-service:8080'
```

Expected result:

```
403
```

19.3 payment-service → inventory-service

Payment should not call Inventory directly in this security model.

```
kubectl exec -n $APP_NS deploy/client-payment -c curl --  
sh -c 'curl -s -o /dev/null -w "%{http_code}\n" http://inventory-service:8080'
```

Expected result:

```
403
```

19.4 outside-mesh → order-service

A non-mesh workload should not call a mesh-protected service.

```
kubectl exec -n $OUTSIDE_NS outside-client --  
curl -sS --max-time 5 http://order-service.${APP_NS}.svc.cluster.local:8080  
|| true
```

Expected result:

```
Connection failure
```

or:

```
upstream connect error
```

or:

```
curl timeout
```

This validates that the mesh is not accepting plaintext traffic from non-mesh clients.

20. Inspect Sidecar Proxies

Check containers in one pod:

```
kubectl get pod -n $APP_NS -l app=order-service -o
jsonpath='{.items[0].spec.containers[*].name}'
echo
```

Expected result:

```
app istio-proxy
```

Describe the pod:

```
kubectl describe pod -n $APP_NS -l app=order-service
```

Look for:

```
Containers:
  app
  istio-proxy
```

View proxy logs:

```
kubectl logs -n $APP_NS deploy/order-service -c istio-proxy --tail=30
```

View application logs:

```
kubectl logs -n $APP_NS deploy/order-service -c app --tail=30
```

21. Troubleshooting Section

21.1 Pods show READY 1/1 instead of 2/2

Problem:

```
Sidecar injection did not happen.
```

Check namespace labels:

```
kubectl get namespace $APP_NS --show-labels
```

Expected:

```
istio.io/rev=asm-x-xx
```

Fix:

```
kubectl label namespace $APP_NS istio.io/rev=$ISTIO_REVISION --overwrite  
kubectl rollout restart deployment -n $APP_NS
```

Wait:

```
kubectl wait --for=condition=Ready pod --all -n $APP_NS --timeout=180s
```

21.2 Allowed traffic returns 403

Check policies:

```
kubectl get authorizationpolicy -n $APP_NS
```

Describe a policy:

```
kubectl describe authorizationpolicy allow-store-front-to-order -n $APP_NS
```

Check the service account name:

```
kubectl get pod -n $APP_NS -l app=client-store-front -o  
jsonpath='{.items[0].spec.serviceAccountName}'  
echo
```

Check that the principal in the policy matches:

```
cluster.local/ns/zt-ecommerce/sa/store-front-sa
```

If your namespace name is different, the principal must also be different.

21.3 Non-mesh traffic still works

Check mTLS policy:

```
kubectl get peerauthentication -n $APP_NS
```

Expected:

```
default
```

Describe it:

```
kubectl describe peerauthentication default -n $APP_NS
```

Expected:

```
mode: STRICT
```

Check that the target service has sidecars:

```
kubectl get pod -n $APP_NS -l app=order-service
```

Expected:

```
READY 2/2
```

21.4 External images cannot be pulled

Check events:

```
kubectl get events -n $APP_NS --sort-by='.lastTimestamp'
```

If you see:

```
ImagePullBackOff  
ErrImagePull
```

validate outbound access from the AKS cluster.

For restricted environments, preload the images into Azure Container Registry and update the manifests to use your ACR image path.

22. Optional: Expose store-front Through Istio Ingress Gateway

This section is optional.

Enable the external Istio ingress gateway:

```
az aks mesh enable-ingress-gateway  
  --resource-group $RESOURCE_GROUP  
  --name $CLUSTER_NAME  
  --ingress-gateway-type external
```

Validate the gateway service:

```
kubectl get svc aks-istio-ingressgateway-external -n aks-istio-ingress
```

Wait until the service has an external IP.

Create a Gateway and VirtualService:

```
cat <<EOF > store-front-gateway.yaml  
apiVersion: networking.istio.io/v1beta1  
kind: Gateway  
metadata:  
  name: store-front-gateway  
  namespace: ${APP_NS}  
spec:  
  selector:  
    istio: aks-istio-ingressgateway-external
```

```

servers:
- port:
    number: 80
    name: http
    protocol: HTTP
  hosts:
  - "*"
---
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: store-front-vs
  namespace: ${APP_NS}
spec:
  hosts:
  - "*"
  gateways:
  - store-front-gateway
  http:
  - route:
    - destination:
        host: store-front.${APP_NS}.svc.cluster.local
        port:
          number: 8080
EOF

```

Apply:

```
kubectl apply -f store-front-gateway.yaml
```

Allow ingress gateway traffic to `store-front`.

First, identify the ingress gateway service account:

```

kubectl get pod -n aks-istio-ingress -l istio=aks-istio-ingressgateway-external
-o jsonpath='{.items[0].spec.serviceAccountName}'
echo

```

Create an allow policy for ingress traffic.

Replace `INGRESS_GATEWAY_SA` if the command above shows a different name.

```

export INGRESS_GATEWAY_SA=$(kubectl get pod -n aks-istio-ingress -l istio=aks-istio-ingressgateway-external
-o jsonpath='{.items[0].spec.serviceAccountName}')

cat <<EOF > allow-ingress-to-store-front.yaml
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: allow-ingress-to-store-front
  namespace: ${APP_NS}
spec:
  selector:
    matchLabels:
      app: store-front
  action: ALLOW
  rules:
  - from:
    - source:
        principals:
        - cluster.local/ns/aks-istio-ingress/sa/${INGRESS_GATEWAY_SA}
EOF

```

Apply:

```
kubectl apply -f allow-ingress-to-store-front.yaml
```

Get the public IP:

```

export INGRESS_HOST_EXTERNAL=$(kubectl -n aks-istio-ingress get service aks-istio-ingressgateway-external
-o jsonpath='{.status.loadBalancer.ingress[0].ip}')

export INGRESS_PORT_EXTERNAL=$(kubectl -n aks-istio-ingress get service aks-istio-ingressgateway-external
-o jsonpath='{.spec.ports[?(@.name=="http2")].port}')

echo "http://${INGRESS_HOST_EXTERNAL}:${INGRESS_PORT_EXTERNAL}"

```

Test:

```
curl -sS "http://${INGRESS_HOST_EXTERNAL}:${INGRESS_PORT_EXTERNAL}"
```

Expected result:


```
store-front: customer entry point
```

23. Optional Advanced Challenge: JWT Validation at the Edge

This section is optional and requires an external identity provider such as Microsoft Entra ID, Auth0, Okta, or another OIDC-compliant provider.

Conceptually, you can use Istio `RequestAuthentication` to validate JWT tokens and `AuthorizationPolicy` to require authenticated requests.

Example structure:

```
apiVersion: security.istio.io/v1
kind: RequestAuthentication
metadata:
  name: store-front-jwt
  namespace: zt-ecommerce
spec:
  selector:
    matchLabels:
      app: store-front
  jwtRules:
    - issuer: "https://login.microsoftonline.com/<TENANT_ID>/v2.0"
      jwksUri: "https://login.microsoftonline.com/<TENANT_ID>/discovery/v2.0/keys"
---
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: require-jwt-store-front
  namespace: zt-ecommerce
spec:
  selector:
    matchLabels:
      app: store-front
  action: ALLOW
  rules:
    - from:
      - source:
          requestPrincipals:
            - "*"

```

Do not apply this policy unless you have a valid JWT available for testing.

This section is useful to explain:

```
mTLS protects service-to-service identity.  
JWT protects end-user or client identity.  
AuthorizationPolicy connects both models.
```

24. Cleanup

Delete the Zero Trust e-commerce namespace:

```
kubectl delete namespace $APP_NS
```

Delete the outside namespace:

```
kubectl delete namespace $OUTSIDE_NS
```

Optional: disable the external Istio ingress gateway:

```
az aks mesh disable-ingress-gateway  
  --resource-group $RESOURCE_GROUP  
  --name $CLUSTER_NAME  
  --ingress-gateway-type external
```

Optional: disable the Istio add-on completely:

```
az aks mesh disable  
  --resource-group $RESOURCE_GROUP  
  --name $CLUSTER_NAME
```

Do not disable Istio if other labs or applications are using the service mesh.

25. Instructor Talking Points

Zero Trust principle

Do not assume that internal traffic is trusted just because it is inside the cluster.

Kubernetes NetworkPolicy vs Istio AuthorizationPolicy

Kubernetes NetworkPolicy controls network reachability at Layer 3 and Layer 4.

Istio AuthorizationPolicy controls service identity and application-level access through the sidecar proxy.

Both can complement each other.

mTLS

mTLS provides encrypted service-to-service traffic and workload identity.

ServiceAccount identity

In Kubernetes and Istio, service accounts become the foundation for workload identity.

Example:

```
cluster.local/ns/zt-ecommerce/sa/order-service-sa
```

This identity can be used in authorization policies.

Default deny

Default deny is one of the most important Zero Trust controls.

Without default deny, accidental access is usually allowed.

Least privilege

Each service should only call the services required by the business workflow.

Example:

```
store-front should call order-service.  
store-front should not call payment-service directly.
```

Validation

A security control is not complete until we validate both sides:

Allowed traffic works.
Denied traffic fails.

26. Expected Final State

At the end of the lab:

- The AKS cluster has Istio enabled.
- The `zt-ecommerce` namespace is part of the mesh.
- All application pods have Istio sidecars.
- mTLS STRICT is enabled.
- Default deny is enabled.
- Only approved service-to-service communication is allowed.
- Unauthorized service calls return `403`.
- Non-mesh traffic cannot call mesh-protected services.

The final security posture is:

Identity-based access
+
Encrypted service-to-service traffic
+
Least-privilege communication
+
Explicit verification

This is the foundation of Zero Trust Security for microservices on AKS.